MitShop: Peer-to-Peer Campus Marketplace

Milestone 1 Project Report & Software Architecture Specification

1. Executive Summary & Objectives

MitShop is a web-based, peer-to-peer campus marketplace tailored specifically for university students. It serves as a centralized platform for buying, selling, and exchanging used academic materials, electronics, and hostel essentials within a single campus ecosystem.

Core Problem

Campus peer-to-peer commerce currently relies on unstructured WhatsApp groups, Discord servers, and Instagram stories. These channels present significant drawbacks:

- **Lack of Organization & Searchability:** Products get buried rapidly in chat feeds without categorisation or status indicators.
- **Safety & Identity Risks:** Public or semi-public groups make it difficult to verify whether a seller or buyer is an actual student living on campus.
- **Transactional Friction:** Price negotiation and meetup logistics occur across fragmented, unstructured messaging threads.

Core Objectives

1. **Campus Authentication:** Enforce student-only access via institute email domain verification.
2. **Structured Catalog:** Offer a searchable listing engine with specific categories, item conditions, and localized hostel/room details.
3. **Contextual Item Chat:** Connect buyers and sellers through direct messaging tied explicitly to a specific product listing.
4. **Local Campus Logistics:** Provide a streamlined cart and order request workflow designed around hostel gate pickups and in-person cash/UPI exchanges.

---

2. Stakeholder Analysis & Survey Findings

A target user survey was conducted across campus to understand current buying habits, safety concerns, and feature priorities.

Survey Metrics

- **Total Responses Collected:** 11
- **Valid Responses (Post-Data Cleaning):** 9 *(2 invalid entries removed)*

Key Findings & Market Insights

- **Primary Pain Points:** Finding active buyers/sellers quickly, awkward price bargaining in public groups, and uncertainty regarding the identity of trading partners.
- **Top Product Categories:** Textbooks & reference manuals, electronics (chargers, monitors, adapters), hostel furniture/appliances, and engineering drawing tools (drafters, lab coats).
- **Verification Demand:** 8 out of 11 respondents highlighted institute email verification (`@campus.edu`) as an essential feature for building trust.
- **Payment & Delivery Preferences:** Preferred payment methods are UPI and Cash on delivery/meetup. Preferred delivery location is in-person handoffs at hostel gates or common campus zones.

Stakeholder Profiles

Stakeholder Role	Needs & Expectations	Platform Impact
Student Seller	Quick listing creation, reaching campus buyers, straightforward communication.	Ability to upload listings with images, manage item availability status, and chat with buyers.
Student Buyer	Simple search/filter, fair pricing, reliable verification of seller identity and location.	Ability to browse by category, filter by hostel location, and initiate item-bound chats.
Campus Context	Low-friction delivery, local pickup points, secure peer transactions.	Standardized pickup locations (e.g., Hostel Gate 1, Main Block) built into order checkout.

---

3. Software Requirement Specification (SRS)

Functional Requirements

User Authentication

- Users must register and log in using a valid campus email address.
- The system shall generate a JSON Web Token (JWT) upon successful authentication to secure subsequent API calls.

Item Listing & Catalog

- Sellers can create listings with title, description, category, condition, price, and primary hostel location.
- Users can view, search, and filter the catalog by category, price range, and item status (Available, Pending, Sold).

Cart & Order Placement

- Buyers can add items to a shopping cart and proceed to order placement.
- Order placement specifies the target meetup location (e.g., specific hostel gate) and preferred payment method (UPI / Cash).

Item-Linked Direct Chat

- The system must allow buyers to initiate a direct chat thread with a seller.
- Every message thread must be linked to a specific `item_id` to maintain contextual transaction logs.

Order Tracking

- Orders transition through defined statuses: `Pending` → `Completed` or `Cancelled`.
- Sellers have authority to mark an order as `Completed` once the physical item handoff occurs.

Non-Functional Requirements

- **Performance:** Frontend page initial load under 1.5 seconds on standard campus Wi-Fi/4G. REST API response time under 200ms for standard read queries.
- **Usability:** Mobile-first, responsive web interface catering to students accessing the platform via smartphones.
- **Security:** Passwords hashed using `bcrypt` prior to database storage. All API endpoints protected via JWT validation middleware and input sanitization.

---

4. System Architecture

MitShop follows a clean **3-Tier Web Architecture** prioritizing simplicity, maintainability, and standard web standards.+--+

| CLIENT TIER |

| React / Next.js Responsive Web Application |

+--+

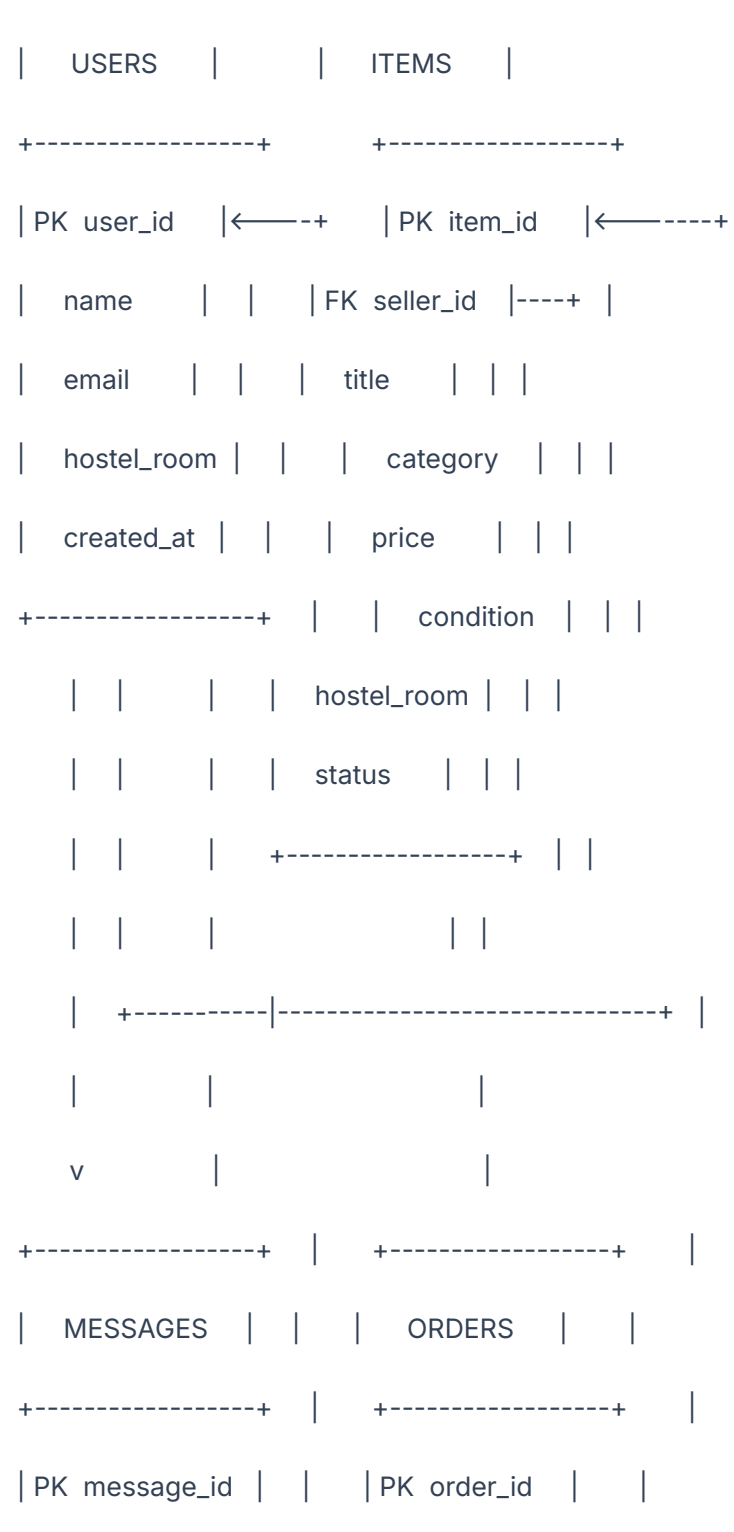


1. **Client Tier (Frontend):** React / Next.js rendering dynamic components, state management for cart/chat, and mobile-responsive UI.
2. **Server Tier (Backend):** Node.js and Express handling business logic, authentication middleware, REST API endpoint routing, and WebSocket connections for real-time messaging.
3. **Database Tier (Data Store):** PostgreSQL or MySQL relational database enforcing strict data schema integrity across users, items, transactions, and chat records.

---

5. Entity-Relationship Data Schema (ERD)

The database design relies on a relational structure ensuring referential integrity across users, inventory, orders, and messages.



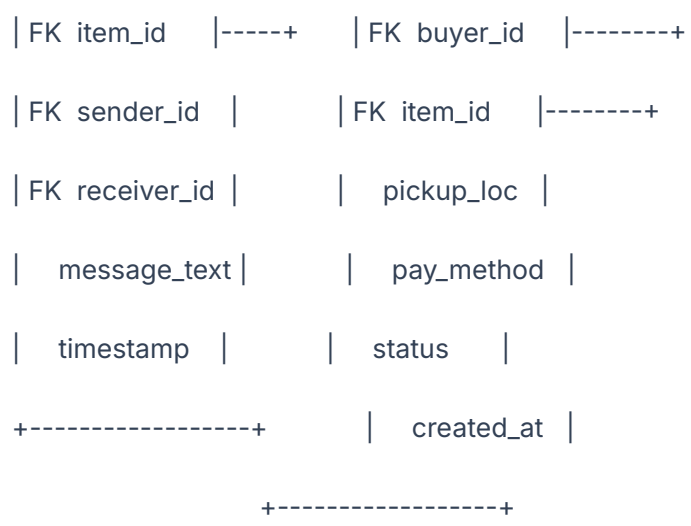


Table Definitions

1. Users

Stores verified student profile details.

- `user_id` (INT, Primary Key, Auto Increment)
- `name` (VARCHAR 100, Not Null)
- `email` (VARCHAR 100, Unique, Not Null)
- `hostel_room` (VARCHAR 50, Not Null)
- `created_at` (TIMESTAMP, Default CURRENT_TIMESTAMP)

2. Items

Stores product listings created by student sellers.

- `item_id` (INT, Primary Key, Auto Increment)
- `seller_id` (INT, Foreign Key → `Users.user_id`)
- `title` (VARCHAR 150, Not Null)
- `category` (VARCHAR 50, Not Null)
- `price` (DECIMAL(10,2), Not Null)
- `condition` (VARCHAR 50, Not Null)
- `hostel_room` (VARCHAR 50, Not Null)
- `status` (ENUM: `'Available'`, `'Pending'`, `'Sold'`, Default `'Available'`)

3. Orders

Tracks purchase requests and pickup logistics.

- `order_id` (INT, Primary Key, Auto Increment)
- `buyer_id` (INT, Foreign Key → `Users.user_id`)
- `item_id` (INT, Foreign Key → `Items.item_id`)
- `pickup_location` (VARCHAR 100, Not Null)
- `payment_method` (ENUM: `'UPI'`, `'Cash'`, Not Null)
- `status` (ENUM: `'Pending'`, `'Completed'`, `'Cancelled'`, Default `'Pending'`)
- `created_at` (TIMESTAMP, Default CURRENT_TIMESTAMP)

4. Messages

Stores context-bound direct communication between buyers and sellers.

- `message_id` (INT, Primary Key, Auto Increment)
- `item_id` (INT, Foreign Key → `Items.item_id`)
- `sender_id` (INT, Foreign Key → `Users.user_id`)
- `receiver_id` (INT, Foreign Key → `Users.user_id`)
- `message_text` (TEXT, Not Null)
- `timestamp` (TIMESTAMP, Default CURRENT_TIMESTAMP)

---

6. Core REST API Endpoints

Resource	HTTP Method	Endpoint	Description	Auth Required
Auth	POST	/api/auth/register	Register new user with campus email	No
Auth	POST	/api/auth/login	Authenticate user & return JWT token	No
Items	GET	/api/items	Fetch all available listings (with filters)	No
Items	POST	/api/items	Create a new item listing	Yes
Items	GET	/api/items/:id	Fetch detailed view of a specific item	No
Orders	POST	/api/orders	Place a new purchase order	Yes

Resource	HTTP Method	Endpoint	Description	Auth Required
Orders	GET	/api/orders	Fetch user's order history (buyer/seller)	Yes
Chat	GET	/api/chat/:itemId	Retrieve chat message history for an item	Yes
Chat	POST	/api/chat/:itemId	Send a new message regarding an item	Yes

---

7. Current Progress & Known Issues

Completed Frontend Interfaces (Milestone 1)

- **Buy Feed View:** Functional product display grid with category filters, search bar, and item status badges.
- **Sell Form View:** Interactive form capturing item title, category, price, condition, hostel block, and photo uploads.
- **Item Chat UI:** Dedicated message interface displaying seller-buyer conversation thread anchored to product context.
- **Cart & Checkout Screen:** Cart drawer and checkout interface supporting hostel gate selection and payment mode preferences.

Known Issue

- **State-Echo / Bot-Repeat Bug in Chat Component:** In the current frontend prototype, sending a chat message triggers an immediate echo of the user's message as an incoming payload. This is due to a temporary mock event listener in the local React component state that duplicates local state updates before real backend WebSocket synchronization is established.

Implementation Plan for Milestone 2

1. **Backend Development:** Build Node.js + Express REST API server structured according to Section 6.
2. **Database Integration:** Setup PostgreSQL instance, execute schema migrations, and connect via ORM/query builder.
3. **Chat State Refactoring:** Replace frontend state mocks with WebSocket connections, resolving the state-echo bug.

4. **End-to-End Testing:** Validate full flow from registration to item listing, chatting, checkout, and order status updates.